Task 2A: Baseline LRU vs. I-RRIP

CS683

October 15, 2026

1 Experimental setup

This report compares the baseline least-recently-used (LRU) cache replacement policy with the implemented instruction-aware RRIP (I-RRIP) policy on three traces. The reported metrics are instructions per cycle (IPC), speedup relative to the baseline, total L2 cache load misses per thousand instructions (L2C load MPKI), L2 instruction-load MPKI, and L2 data-load MPKI.

For each trace, speedup is calculated as

$$\text{Speedup} = \frac{\text{IPC}_{\text{I-RRIP}}}{\text{IPC}_{\text{Baseline LRU}}}.$$

The baseline speedup is therefore 1.0 for every trace. MPKI values are reproduced from the supplied results CSV. Lower MPKI indicates fewer load misses per thousand instructions; IPC and speedup are better when higher.

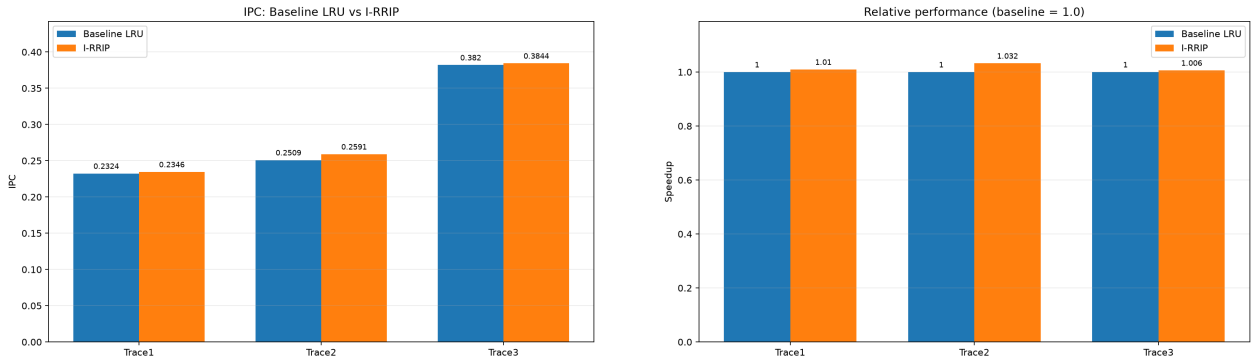
2 Measured results

Table 1: IPC, speedup, and L2 load MPKI for each trace and policy.

Trace	Policy	IPC	Speedup	L2C load MPKI	Instruction MPKI	Data MPKI
Trace 1	Baseline LRU	0.232386	1.000000	17.7506	10.3965	7.3541
	I-RRIP	0.234612	1.009579	19.0635	9.0358	10.0277
Trace 2	Baseline LRU	0.250946	1.000000	28.2597	19.6990	8.5607
	I-RRIP	0.259057	1.032322	22.5938	10.1630	12.4308
Trace 3	Baseline LRU	0.382022	1.000000	3.9686	1.1354	2.8332
	I-RRIP	0.384399	1.006222	3.9708	0.6992	3.2716

2.1 Plots

Figure 1 shows IPC and speedup. Figures 2–4 show the total L2C load MPKI and its instruction/data components.



(a) IPC

(b) Speedup over baseline LRU

Figure 1: Performance of baseline LRU and I-RRIP.

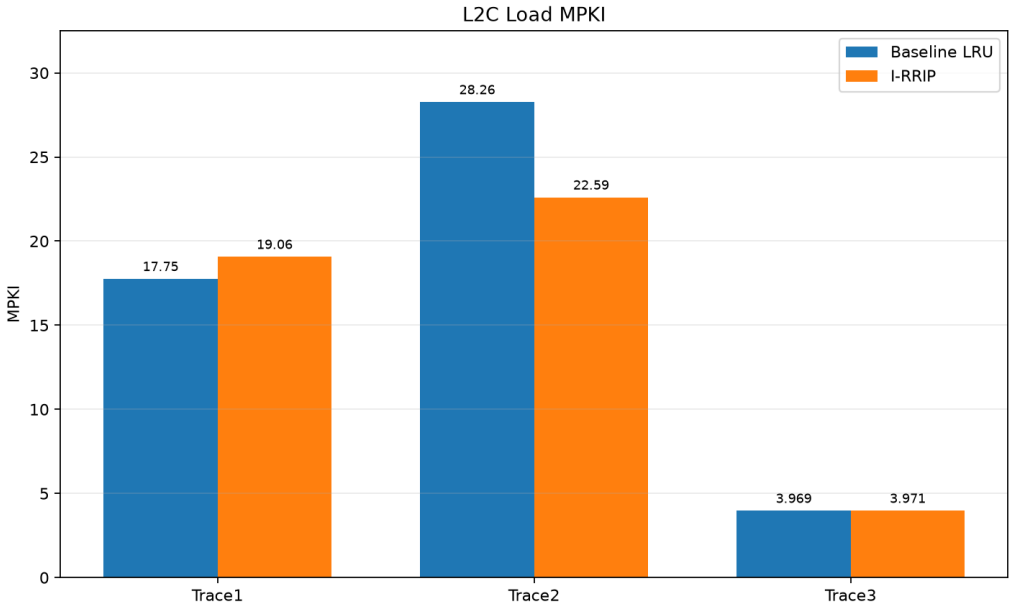


Figure 2: Total L2 cache load MPKI.

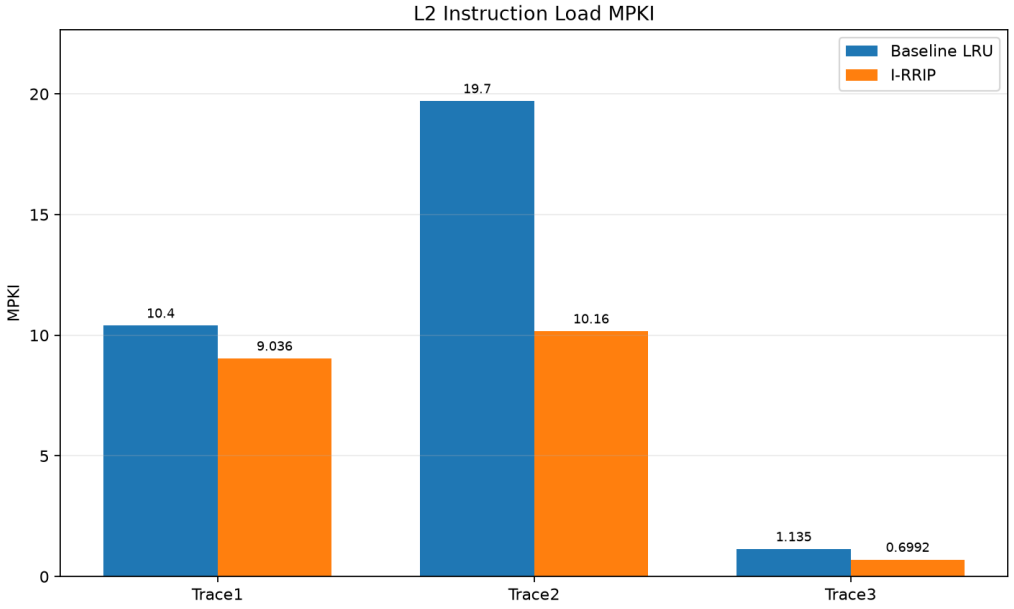


Figure 3: L2 instruction-load MPKI.

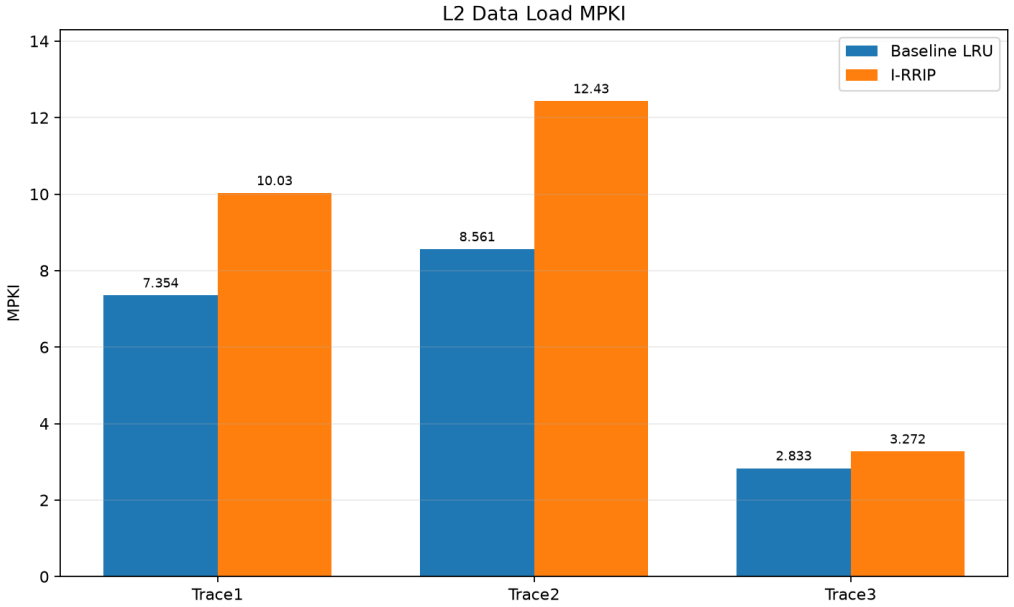


Figure 4: L2 data-load MPKI.

3 I-RRIP design rationale

The I-RRIP policy is intended to protect instruction-cache lines in the shared L2 cache from being displaced too aggressively by data lines. The implementation makes instruction lines more resistant to eviction, while continuing to use re-reference prediction values (RRPVs) to distinguish lines that are likely to be reused soon from those predicted to be reused farther in the future.

3.1 Insertion

On insertion, an instruction line is assigned RRPV 0, while a data line is assigned RRPV 2. A lower RRPV represents a line predicted to be reused sooner; the maximum RRPV represents a line

predicted to be distant from reuse. Giving instruction lines RRPV 0 gives them a strong initial protection after they enter the cache. Data lines enter with a less-protected value, so they can be selected for replacement earlier when the cache is under pressure. This choice prioritizes instruction retention without making data lines permanently ineligible for eviction.

3.2 Promotion on a hit

A cache hit promotes the accessed line by setting its RRPV to 0. This records evidence that the line has been reused and should be retained. Applying this promotion to both instruction and data lines preserves the benefit of temporal locality for both types of accesses. The policy does not ignore repeatedly accessed data lines merely because instruction lines receive preferential insertion and eviction treatment.

3.3 Eviction

When selecting a victim, the implementation skips maximum-RRPV instruction lines if at least one data line is available as a candidate. Otherwise, it uses the normal RRIP victim-selection procedure: choose a line at the maximum RRPV, increment RRPVs as needed when no such line exists, and repeat the search. This makes instruction lines less likely to be evicted while still allowing them to be replaced when the alternatives do not provide a suitable victim. The fallback to RRIP aging also ensures that victim selection can make progress rather than protecting instruction lines unconditionally.

3.4 Why this combination?

The insertion, promotion, and eviction rules work together. Preferential insertion gives instruction lines an initial advantage; hit promotion protects lines of either type when they demonstrate reuse; and victim selection applies the instruction preference only when a data-line alternative exists. This is a compromise between protecting instruction working sets and allowing data lines to compete for cache capacity. A policy that always protected instruction lines could increase data misses, while treating both line types identically would lose the intended instruction-retention benefit.

4 Discussion of results

The measured results show that I-RRIP improves IPC on all three traces, but the size of the gain varies:

- **Trace 1:** IPC increases from 0.232386 to 0.234612, a speedup of 1.009579× (about 0.96%). Instruction MPKI falls from 10.3965 to 9.0358, but data MPKI rises from 7.3541 to 10.0277. The increase in data misses outweighs the instruction-miss reduction in the aggregate L2C load MPKI, which rises from 17.7506 to 19.0635.
- **Trace 2:** IPC increases from 0.250946 to 0.259057, the largest gain, with a speedup of 1.032322× (about 3.23%). Total L2C load MPKI decreases from 28.2597 to 22.5938, and instruction MPKI falls substantially. Data MPKI increases, showing the trade-off between instruction and data behavior, but the aggregate load MPKI improves.
- **Trace 3:** IPC increases slightly, giving a speedup of 1.006222× (about 0.62%). Instruction MPKI decreases from 1.1354 to 0.6992, while data MPKI increases from 2.8332 to 3.2716. Total L2C load MPKI is nearly unchanged, increasing slightly from 3.9686 to 3.9708.

Overall, I-RRIP improves IPC on all three traces and reduces instruction-load MPKI in every case. However, its impact on data-load MPKI is negative for all three traces, and total L2C load MPKI

does not improve for Trace 1 or Trace 3. These results illustrate why instruction-aware replacement is a trade-off rather than a universal reduction in every miss metric. IPC is the final performance measure, and the results show that a reduction in instruction misses can coexist with an increase in data misses.

5 Conclusion

Compared with baseline LRU, the implemented I-RRIP policy achieves speedups of 1.009579×, 1.032322×, and 1.006222× on Traces 1, 2, and 3, respectively. The strongest result is on Trace 2, where IPC improves by approximately 3.23% and total L2C load MPKI falls by about 20.1%. The policy consistently lowers instruction-load MPKI, but the corresponding rise in data-load MPKI means that aggregate miss behavior depends on the trace. The measurements support I-RRIP as a useful instruction-aware policy for these runs, while also showing that its benefit is workload-dependent.